Clustering Binary Data Optimizing the Exclusiveness of Features

by J. Orestes Cerdeira and Tiago Monteiro-Henriques

Abstract A problem that has been studied for a long time in Vegetation Science consists in finding distinct groups among a collection of vegetation samples (relevés). Recently a criterion has been proposed to capture the patterns of differential species among the groups from any arbitrary M-cluster of relevés. The criterion optimization is quite complex (NP-hard) as it implies searching a huge set of possible combinations of relevés into M groups. In this paper we give an integer linear programming formulation to optimize the criterion for M=2, that can be used to solve moderately-sized data problems; we also describe a greedy randomized adaptive search procedure and a simulated annealing algorithm for arbitrary M. We combined these different approaches and prepared a collection of software functions, which are available in the open-source R package diffval. We illustrate the use of such functions using real-world data.

1 Introduction

Vegetation classification is a complex problem that has occupied researchers for decades. This is traditionally done by manipulating matrices of vegetation samples (relevés), as described in detail by Mueller-Dombois and Ellenberg (1974). The main purpose of this manipulation is clustering, i.e., the researcher seeks to partition the collection of relevés in groups (subsets of relevés), which can be distinguished floristically (by subsets of the present taxa). Such manipulation is even more ambitious and seeks to find, simultaneously, an optimal number of clusters to partition the relevé set. This process is usually called tabulation, tabular classification, or table sorting. Manual tabulation is a particularly tedious process, being impractical for large data sets.

With the advent of computers and software, several authors have proposed a plethora of different approaches to vegetation classification. In essence, all of them aim at clustering groups of relevés based on common taxa that are expected to differentiate those groups, ultimately pursuing a typical block-structured sorted table. The vast majority of such approaches does this heuristically, relying on numerical clustering methods based on pairwise (dis)similarity measures. Direct optimization, even if considered a natural way of approaching the issue, is hindered by the huge number of possible combinations of relevé groups (Peet and Roberts, 2013). However, there are some proposals trying to find directly the desired block-structured tables, e.g., Two-Way Indicator Species Analysis–TWINSPAN (Hill, 1979), analysis of concentration (Feoli and Orlóci, 1979), chi-squared optimization (Podani and Feoli, 1991), the ESPRESSO software (Bruelheide and Flintrop, 1994), the COCKTAIL algorithm (Bruelheide, 2000), and modified-TWINSPAN (Roleček et al., 2009). With the same objective Monteiro-Henriques (2025) recently proposed an optimization approach to tabulation, devising a new index specifically for that purpose: the DiffVal index. In Monteiro-Henriques (2025) comparisons to other clustering methods, using both synthetic data sets with known group structure, and real-world data are provided.

In general, the approaches mentioned for classifying vegetation data fall within the clustering or partitioning (Kaufman and Rousseeuw, 1990) and biclustering (Castanho et al., 2024) frameworks. Within these broader disciplines, what are here called *relevés* are usually referred to as *objects* or *samples*, and *species* as *features*, *variables*, or *attributes*.

Given a vegetation table and a partition of the relevés, the DiffVal index quantifies the ability of a taxon in differentiating those groups, taking into account the following characteristics:

C1) The greater the relative frequency in the group (or groups) where the taxon occurs, the higher is the corresponding DiffVal. This is a commonly sought feature among diagnostic taxa (see, e.g., the IndVal index of Dufrêne and Legendre, 1997), as frequent taxa in the group have great bioindicator and practical value in recognizing communities in the field.

C2) The greater the exclusiveness of a taxon to a group (or groups) the better, i.e., the greater the full absence from other groups, the higher is the corresponding DiffVal. This relates to fidelity (sensu Mueller-Dombois and Ellenberg, 1974).

C3) Conversely, a taxon occurring in all groups should present a zero DiffVal, as it is useless in distinguishing the groups with its presence.

Balancing those characteristics (C1 to C3) in a single index is challenging, however, the DiffVal index is able to incorporate them in a simple combinatorial way (i.e. counting the presences, the absences and the group sizes). Eventually, having the DiffVals for all the taxa in a vegetation table, we

can compute their sum, which is a natural criterion to portray the overall pattern of present differential taxa. Finding the desired block-structured table will be possible then, if we are able to find relevé partitions that maximize this sum.

Mathematically, this contextualizes as follows. (For an ecological interpretation see Monteiro-Henriques (2025), where a worked example of DiffVal calculation with a small hypothetical vegetation table is provided.) The presence of taxa in relevés is described as a, $m \times n$, matrix $A = [a_{ij}]$, with 0/1 entries

$$a_{ij} = \begin{cases} 1 & \text{if taxon } i \text{ occurs in relevé } j \\ 0 & \text{otherwise} \end{cases}$$

In what follows we assume that $\sum_{j=1}^{n} a_{ij} \geq 1$, i.e., any taxon occurs at least in one relevé.

A solution S_M for the vegetation classification in M groups is an M-partition of the set of columns (relevés) C: $G_1, \ldots, G_M$ such that $\cup_{k=1}^{M} G_k = C$ and $G_k \cap G_l = \emptyset$, with $k \neq l = 1, \ldots, M$.

The value of the DiffVal index for taxon i w.r.t. solution S_M is

$$\text{DiffVal}_i(S_M) = \sum_{k=1}^{M} \frac{\sum_{j=1}^{n} a_{ij} x_j^k}{\sum_{j=1}^{n} x_j^k} \frac{\sum_{l\neq k=1}^{M} \sum_{j=1}^{n} x_j^l g_i^l}{\sum_{l\neq k=1}^{M} \sum_{j=1}^{n} x_j^l} \frac{1}{M - \sum_{l=1}^{M} g_i^l} \tag{1}$$

where $x_j^k = 1$ ($x_j^k = 0$), if column j is (not) in group G_k of solution S_M and $g_i^k = 1$ ($g_i^k = 0$) if, in group G_k of S_M, (not) all entries $a_{ij} = 0$.

Thus, expressions $\sum_{j=1}^{n} a_{ij} x_j^k$ and $\sum_{j=1} x_j^k$ indicate the number of occurrences of taxon i in group G_k and the size of group G_k, respectively, and, therefore $\frac{\sum_{j=1}^{n} a_{ij} x_j^k}{\sum_{j=1} x_j^k}$ is the relative representation of taxon i in G_k, incorporating the characteristic C1, referred above, in the DiffVal index.

Expression $\frac{\sum_{l\neq k=1}^{M} \sum_{j=1}^{n} x_j^l g_i^l}{\sum_{l\neq k=1}^{M} \sum_{j=1}^{n} x_j^l}$ captures the absence of taxon i in all groups but G_k. The numerator $\sum_{l\neq k=1}^{M} \sum_{j=1}^{n} x_j^l g_i^l$ sums the sizes of all empty groups, while $\sum_{l\neq k=1}^{M} \sum_{j=1}^{n} x_j^l$ is the sum of the sizes of all groups except G_k. Thus, the ratio $\frac{\sum_{l\neq k=1}^{M} \sum_{j=1}^{n} x_j^l g_i^l}{\sum_{l\neq k=1}^{M} \sum_{j=1}^{n} x_j^l}$ is the proportion of emptiness of taxon i outside G_k group, incorporating characteristic C2 in the index. Note that if taxon i occurs in all groups, i.e., $g_i^k = 0$, for $k = 1, \ldots, M$, then $\text{DiffVal}_i(S_M) = 0$ fulfilling characteristic C3.

Finally, $M - \sum_{l=1}^{M} g_i^l$ in the denominator of (1) is used to make the index DiffVal ≤ 1.

Taking the sum of $\text{DiffVal}_i(S_M)$, for all taxon i, we get

$$\text{TotDiffVal}(S_M) = \sum_{i=1}^{m} \text{DiffVal}_i(S_M) \tag{2}$$

that quantifies the extent to which solution S_M is based on differential taxa, that we aim to maximize. Dividing $\sum_{i=1}^{m} \text{DiffVal}_i(S_M)$ by m also ensures a value bounded between 0 and 1, yet this is unnecessary for the maximization, thus it is disregarded here.

The following constraints ensure that variables x_j^k have the meaning given above (i.e., $x_j^k = 1$ if column j is in group G_k, and $x_j^k = 0$ otherwise) and that, in every solution that maximizes (2), variables g_i^k also have the meaning above (i.e., $g_i^k = 1$ if taxon i is absent from all relevés of group G_k, and $g_i^k = 0$, otherwise).

$$\sum_{k=1}^{M} x_j^k = 1, \qquad j = 1, \ldots, n \tag{3}$$

$$\sum_{j=1}^{n} x_j^k \geq 1, \qquad k = 1, \ldots, M \tag{4}$$

$$x_j^k \in \{0, 1\} \qquad j = 1, \ldots, n; k = 1, \ldots, M \tag{5}$$

$$g_i^k \leq 1 - a_{ij} x_j^k, \qquad j = 1, \ldots, n; k = 1, \ldots, M; i = 1, \ldots, m \tag{6}$$

Constraints (5) define the range of variables x_j^k. Equation (3) assigns each column j to exactly one group, and inequality (4) ensures that every group has at least one column. Hence, variables x_j^k define an M-partition of the set of columns of matrix A.

Inequalities (6) define upper bounds on variable g_i^k: $g_i^k \leq 0$ when taxon i is present in some relevé j of group G_k (i.e., $a_{ij}x_j^k = 1$), and $g_i^k \leq 1$ when taxon i is absent from group G_k (i.e., $a_{ij}x_j^k = 0$, for all j). Clearly, the upper bounds will be attained in every M-partition that maximizes the value of TotDiffVal (2).

Maximizing (2) subject to (3)-(6) is an NP-hard problem (Garey and Johnson, 1979), for any fixed number of groups $M \geq 2$. We briefly sketch the proof. For the case $M = 2$, we reduce from the minimum graph bisection problem, which is known to be NP-hard (Garey et al., 1976). In that problem, the task is to divide the vertices of a graph into two equal parts while minimizing the number of edges crossing between them. This is closely related to our setting: a minimum bisection can be reformulated as finding a balanced 2-partition of the transposed incidence matrix of the graph that maximizes the TotDiffVal index. We then show that such balanced 2-partitions arise as optimal solutions of TotDiffVal maximization (for $M = 2$) on a suitably constructed larger matrix, without explicitly enforcing the balancing constraint. The full reduction is presented in the Appendix. Intuitively, the problem is computationally intractable because the number of possible partitions grows exponentially with the size of the data, and the balancing requirement further constrains the search. By induction on M, the NP-hardness extends to any fixed number of groups $M \geq 2$ (see Appendix). Hence, unless $P = NP$, no efficient algorithm exists to find an M-partition that maximizes TotDiffVal.

The rest of this paper is organized as follows. In Section 2 we address the problem of maximizing TotDiffVal in the case where $M = 2$, and give two integer linear programming formulations. In Sections 3 and 4 we present a greedy randomized adaptive search procedure and a simulated annealing algorithm to maximize the criterion TotDiffVal for arbitrary M, respectively. The approaches of Sections 2, 3 and 4 are combined in a number of software functions included in the R package **diffval** (Monteiro-Henriques and Cerdeira, 2025) which we describe in Section 5, illustrating their application on real-world data.

2 Case $M = 2$

When $M = 2$ the value of x_j^1 (x_j^2) determines the value of x_j^2 (x_j^1). We may therefore replace variables x_j^1 and x_j^2 by a unique variable x_j, where $x_j = 1$ indicates that column j is assigned to one of the two groups, say, e.g., group G_1. Thus, $x_j = 0$ indicates that column j is assigned to group G_2.

Constraints (3)-(6) may be rewritten as follows.

$$\sum_{j=1}^{n} x_j \geq 1 \tag{7}$$

$$\sum_{j=1}^{n} x_j \leq \left\lfloor \frac{n}{2} \right\rfloor \tag{8}$$

$$x_j \in \{0, 1\} \qquad j = 1, \ldots, n \tag{9}$$

$$g_i^1 \leq 1 - a_{ij}x_j, \qquad j = 1, \ldots, n; i = 1, \ldots, m \tag{10}$$

$$g_i^2 \leq 1 - a_{ij}(1 - x_j), \qquad j = 1, \ldots, n; i = 1, \ldots, m \tag{11}$$

Constraints (7)-(9) define a 2-partition of the set of columns: $x_j = 1$ ($x_j = 0$) if column j is assigned to group G_1 (G_2), while ensuring that none of the two groups is the empty set.

Inequalities (10) and (11) imply that $g_i^k \leq 0$ when taxon i occurs in some relevé j of group G_k, and that $g_i^k \leq 1$ when taxon i is absent from group G_k.

Expression (1), for $M = 2$, reads

$$\frac{\sum_{j=1}^{n} a_{ij}x_j}{\sum_{j=1}^{n} x_j} g_i^2 + \frac{\sum_{j=1}^{n} a_{ij}(1 - x_j)}{n - \sum_{j=1}^{n} x_j} g_i^1$$

Note that the term $(2 - (g_i^1 + g_i^2))^{-1}$ is not needed since (recall that we assumed $\sum_{j=1}^{n} a_{ij} \geq 1$) either $g_i^1 + g_i^2 \leq 1$, case where for every optimal solution we will have $g_i^1 + g_i^2 = 1$ (i.e., taxon i occurs in exactly one of the two groups) and therefore $(2 - (g_i^1 + g_i^2))^{-1} = 1$, or else $g_i^1, g_i^2 \leq 0$, case where for every optimal solution $g_i^1 = g_i^2 = 0$ (i.e., taxon i occurs in the two groups) and therefore

$\frac{\sum_{j=1}^{n} a_{ij}x_j}{\sum_{j=1}^{n} x_j^1}\, g_i^2 + \frac{\sum_{j=1}^{n} a_{ij}x_j}{n-\sum_{j=1}^{n} x_j}\, g_i^1 = 0.$

Taking into account that when $g_i^2 = 1$ ($g_i^1 = 1$) we have $\sum_{j=1}^{n} a_{ij}x_j = \sum_{j=1}^{n} a_{ij}$ ($\sum_{j=1}^{n} a_{ij}(1-x_j) = \sum_{j=1}^{n} a_{ij}$), if we let $\alpha_i = \sum_{j=1}^{n} a_{ij} \geq 1$, the formula above can be written as

$$\frac{\alpha_i}{\sum_{j=1}^{n} x_j}\, g_i^2 + \frac{\alpha_i}{n-\sum_{j=1}^{n} x_j}\, g_i^1$$

Hence, when $M = 2$ the problem of maximizing TotDiffVal (2) subject to (3)-(6) converts to maximizing

$$\sum_{i=1}^{m} \left(\frac{\alpha_i}{\sum_{j=1}^{n} x_j}\, g_i^2 + \frac{\alpha_i}{n-\sum_{j=1}^{n} x_j}\, g_i^1 \right) \tag{12}$$

subject to the linear constraints (7)-(11).

This is a fractional 0/1 programming problem (Borrero et al., 2017) for which we now give two linearization approaches.

2.1 Model 1

The linearization technique that we use here follows the procedure described in Borrero et al. (2017). If we let

$$y_i^1 = \frac{\alpha_i}{\sum_{j=1}^{n} x_j}\, g_i^2 \text{ and } y_i^2 = \frac{\alpha_i}{n-\sum_{j=1}^{n} x_j}\, g_i^1$$

expression (12) reads

$$\sum_{i=1}^{m} (y_i^1 + y_i^2) \tag{13}$$

The two equations above may be written as

$$\sum_{j=1}^{n} y_i^1 x_j = \alpha_i\, g_i^2 \text{ and } n y_i^2 - \sum_{j=1}^{n} y_i^2 x_j = \alpha_i\, g_i^1 \tag{14}$$

that can be linearized by letting $z_{ij}^1 = y_i^1 x_j$ and $z_{ij}^2 = y_i^2 x_j$, writing equations (14) as

$$\sum_{j=1}^{n} z_{ij}^1 = \alpha_i\, g_i^2, \qquad i = 1, \ldots, m \tag{15}$$

$$n y_i^2 - \sum_{j=1}^{n} z_{ij}^2 = \alpha_i\, g_i^1, \qquad i = 1, \ldots, m \tag{16}$$

and relating z_{ij}^k to y_i^k, $k = 1, 2$, and to x_j through the following inequalities

$$z_{ij}^1 \leq x_j, \qquad j = 1, \ldots, n; i = 1, \ldots, m \tag{17}$$

$$z_{ij}^1 \leq y_i^1, \qquad j = 1, \ldots, n; i = 1, \ldots, m \tag{18}$$

$$z_{ij}^1 \geq y_i^1 + x_j - 1, \qquad j = 1, \ldots, n; i = 1, \ldots, m \tag{19}$$

$$z_{ij}^2 \leq x_j, \qquad j = 1, \ldots, n; i = 1, \ldots, m \tag{20}$$

$$z_{ij}^2 \leq y_i^2, \qquad j = 1, \ldots, n; i = 1, \ldots, m \tag{21}$$

$$z_{ij}^2 \geq y_i^2 + x_j - 1, \qquad j = 1, \ldots, n; i = 1, \ldots, m \tag{22}$$

$$z_{ij}^1, z_{ij}^2 \geq 0, \qquad j = 1, \ldots, n; i = 1, \ldots, m \tag{23}$$

Since x_j are 0/1 variables, inequalities (17), (23) and (18), (19) imply that $z_{ij}^1 = y_i^1 x_j$, and (15) becomes equivalent to the left equation in (14). A similar argument can be used to show that $z_{ij}^2 = y_i^2 x_j$

and (16) is equivalent to the right equation in (14).

Hence, the problem of maximizing (12) subject to (7)-(11) can be formulated as the mixed integer linear programming that consists in maximizing (13) subject to (7)-(11), (15)-(23). We call this Model 1.

2.2 Model 2

We call Model 2 the model that results from fixing the size of group G_1, i.e., requiring that

$$\sum_{j=1}^{n} x_j = t \tag{24}$$

and writing the objective function as follows.

$$M2(t) = \sum_{i=1}^{m} \left(\frac{\alpha_i}{t}\, g_i^2 + \frac{\alpha_i}{n-t}\, g_i^1 \right) \tag{25}$$

In that way, the problem consists of finding, for $t = 1, \ldots, \lfloor \frac{n}{2} \rfloor$, the maximum of $M2(t)$, subject to (24), (9)-(11).

In Section 5 we report some results on the computational performance of the two models.

3 A greedy randomized adaptive search procedure

The greedy algorithm for a combinatorial maximization problem starts with set $S = \emptyset$, and at each step chooses an element to add to S that most increases the objective function and that will not turn S infeasible. The algorithm stops when no more elements on these conditions exist.

The greedy randomized adaptive search procedure (GRASP, see Feo and Resende, 1995; Festa and Resende, 2018) chooses the elements to add to current S with probabilities which are proportional to the values they add to the objective function.

Our GRASP starts by uniformly selecting M columns of matrix A (relevés), and by assigning each column to exactly one of the M groups. If $j_1, \ldots, j_M$ are the indices of the selected columns, the incidence vector of the starting set S is $x_{j_1}^1 = \cdots = x_{j_M}^M = 1$ and $x_j^k = 0$, for every $j \neq j_1, \ldots, j_M$ and $k = 1, \ldots, M$.

At each step a column is chosen to be added to a certain group of current S in the following way. For every index j of a column not in S, i.e., $x_j^k = 0$, $k = 1, \ldots, M$, we define S_j^k to be the (partial) solution resulting from adding to group k of S the column of index j, i.e., the incidence vector x of S_j^k is the same as S, except that, for S_j^k, $x_j^k = 1$. We then compute $\text{TotDiffVal}(S_j^k) = \sum_{i=1}^{m} \text{DiffVal}_i(S_j^k)$, where $\text{DiffVal}_i(S_j^k)$ is given by expression (1), if $M - \sum_{l=1}^{M} g_i^l \neq 0$ and $\text{DiffVal}_i(S_j^k) = 0$, if $M - \sum_{l=1}^{M} g_i^l = 0$. The new current set S is randomly chosen with probabilities proportional to the values of $\text{TotDiffVal}(S_j^k)$, among the p% sets S_j^k having the largest values of $\text{TotDiffVal}(S_j^k)$.

The algorithm stops when current set S includes all columns of A, i.e., when equation (3) is satisfied. We call S the GRASP solution.

4 A simulated annealing algorithm

At each step, the simulated annealing (SA) algorithm (Kirkpatrick et al., 1983; Aarts et al., 1997) randomly selects some *neighbour* solution S' of the current feasible solution S, and probabilistically decides whether to maintain S as current or to replace S by S'.

In our SA implementation the neighbourhood of a solution S is the set of all feasible solutions obtained by moving one column from one group to another group of S. At iteration i, a neighbour S' of current S is uniformly chosen and S' replaces the current solution S if $\Delta_i = \text{TotDiffVal}(S') - \text{TotDiffVal}(S) \geq 0$ or, with probability $exp(\Delta_i / T_i)$, if $\Delta_i < 0$, where $T_i > 0$ is the *temperature* at iteration i. We start with a given initial temperature $T_1 \leq 1$, and gradually decrease the current value by α% every nt iterations, so that in the last iteration the temperature reaches a given established value. The algorithm stops with the best solution found at any stage, after a given number nit of iterations.

Table 1: Running time, memory allocation and objective values for the two formulations, applied to the *T. baccata* forests data set.

formulation	total time	memory allocation	TDV	Gurobi returned status
Model 1	1.69m	9.57GB	0.1513158	"TIME_LIMIT"
Model 2	5.02s	118.02MB	0.1513158	"OPTIMAL" (in all calls)

5 Implementation in R

The optimization models and the algorithms proposed in the previous sections were implemented in R language and included in the R package **diffval**, specifically in three functions: `optim_tdv_gurobi_k_2()`, `partition_tdv_grasp()`, and `optim_tdv_simul_anne()`. As mentioned in Section 1, dividing TotDiffVal by m ensures a value bounded between 0 and 1. In the implemented R functions we make use of that normalization, and we call TDV TotDiffVal/m. Package **diffval** contains a collection of functions to find, visualize and explore patterns of differential taxa in vegetation data (namely, in a phytosociological table) using the DiffVal and the TDV.

We will illustrate the use of the implemented functions with a real-world data set of *Taxus baccata* forests (from Portela-Pereira et al., 2021), which is also included in the package. The data set consists of a binary (presence/absence) phytosociological matrix (209 rows and 33 columns) containing relevés of *T. baccata* forests, from the north-west of the Iberian Peninsula. Each column corresponds to a phytosociological relevé and each row corresponds to a taxon. In the matrix, presences are recorded as 1 and absences as 0. We start by loading the package and the data set:

```
library(diffval)
data("taxus_bin", package = "diffval")
```

Function `optim_tdv_gurobi_k_2()` implements the linear programming formulations (Model 1 and Model 2) given in Section 2, calling Gurobi (Gurobi Optimization, LLC, 2023), a well-known integer programming solver. Package **prioritizr** (Hanson et al., 2022) contains a comprehensive vignette (*Gurobi Installation Guide*), which can guide the user through the process of obtaining a license, installing the Gurobi optimizer, activating the license and eventually installing the R package **gurobi**.

In the R implementation, Models 1 and 2 are referred to as `"t-independent"` and `"t-dependent"`, respectively. By default, a time limit of 5 seconds (`time_limit = 5`) is set for each call of Gurobi, as computation time can become long for large matrices. Note that while in the `"t-independent"` formulation Gurobi is called only once, in the `"t-dependent"` formulation Gurobi is called $\lfloor \frac{n}{2} \rfloor$ times.

For the `"t-independent"` formulation the function returns the solution produced by Gurobi, which is an optimal 2-partition if the set time limit is not exceeded. If the time limit is exceeded, optimality is not ensured.

For the `"t-dependent"` formulation, the function returns the best 2-partition among the $\lfloor \frac{n}{2} \rfloor$ solutions produced by Gurobi. If the set time limit is not exceeded in all the Gurobi calls, the returned 2-partition is optimal. Otherwise, optimality is not ensured.

After comparing the computational performances of the two models, we set Model 2 as the default formulation (`formulation = "t-dependent"`). This is illustrated in Table 1, using the *T. baccata* forests data set. Running time and memory allocation were obtained with function `mark()` from package **bench** (Hester and Vaughan, 2021). In this comparison, we set a time limit of 80 seconds for the Gurobi call in Model 1 and 5 seconds for each of the 16 Gurobi calls in Model 2. Model 2 clearly outperformed Model 1, being faster and having a lower memory allocation burden. Notice that both models returned the same TDV yet Model 1 did not return a final optimal status, indicating that the process was broken by the imposed time limit.

For the *T. baccata* forests data set, the function `optim_tdv_gurobi_k_2()` returns a 2-partition which isolates one relevé (`"LE01"`):

```
result <- optim_tdv_gurobi_k_2(taxus_bin)
all(result$status.runs == "OPTIMAL")

#> [1] TRUE

result$par

#>  [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1

result$objval
```

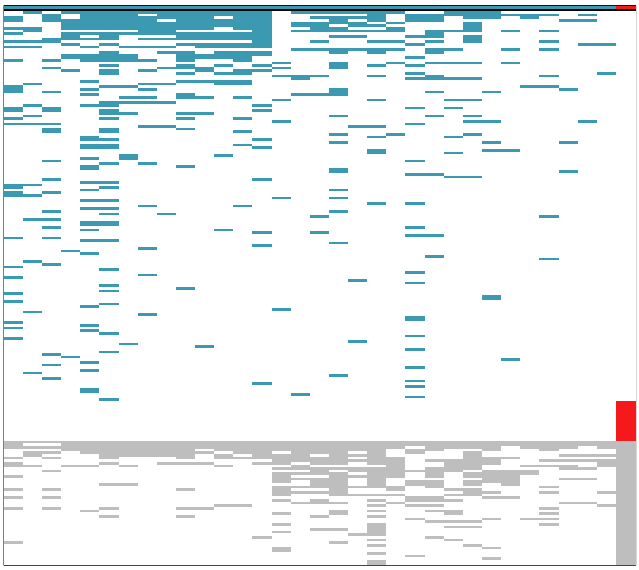

Figure 1: Image of the sorted phytosociological matrix showing the isolated relevé. Each coloured rectangle represents a presence in the matrix. Presences of taxa exclusive of group 1 are represented in blue, while those exclusive of group 2 are in red. Taxa shared by the two groups are represented in grey.

```
#> [1] 0.1513158
```

The isolated relevé stands out for the high number of taxa (62, while the mean number of taxa is 29.7), from which 15 taxa are exclusive to it. Using function `tabulation()` from **diffval**, it is possible to visualize the sorted phytosociological table, given the obtained partition of its columns:

```
tabul <- tabulation(
 m_bin = taxus_bin,
 p = result$par,
 taxa_names = rownames(taxus_bin),
 plot_im = "normal",
 palette = "Zissou 1"
)
```

The `tabulation()` function reorders the columns of the original matrix by putting together the relevés belonging to the same group (as defined by the given partition). Additionally, it reorders the rows, by placing exclusive taxa towards the top of the table, while the taxa occurring in all groups of the partition are greyed out and placed towards the bottom of the matrix. Figure 1 shows the image generated by the `tabulation()` function using as input the 2-partition previously obtained using Gurobi. It is possible to distinguish the isolated relevé, showing its exclusive taxa in red.

For bigger matrices, where `optim_tdv_gurobi_k_2()` computation time might become prohibitive, heuristics such as `partition_tdv_grasp()` or `optim_tdv_simul_anne()`, or a combination of both, are recommended to find patterns of differential species.

Function `partition_tdv_grasp()` creates partitions of the data set using the greedy randomized adaptive search procedure described in Section 3. The input parameter `thr` (from 0 to 1) defines the p% sets defined in Section 3. For `thr = 1` the algorithm corresponds to the greedy algorithm. By default, `thr = 0.95`. Exemplifying with the *T. baccata* forests data set (using `set.seed()` only to ensure the reproducibility of the outputs):

```
set.seed(1)
partition_tdv_grasp(taxus_bin, 3)
```

```
#>  [1] 3 2 3 1 2 2 2 2 2 2 1 1 2 1 1 1 1 1 1 1 1 3 1 1 1 1 1 1 1 1 1 1
```

```
# [1] 3 2 3 1 2 2 2 2 2 2 1 1 2 1 1 1 1 1 1 1 1 3 1 1 1 1 1 1 1 1 1 1
```

Function `optim_tdv_simul_anne()` implements the simulated annealing algorithm as presented in Section 4, searching for a k-partition (k, defined by the user), optimizing the TDV, i.e., searching

for a global maximum of TDV (by rearranging the relevés into k groups). As initial partition, the function can use any user-given partition (`p_initial`), a uniformly generated partition (`p_initial = "random"`) or (if `use_grasp = TRUE`) a partition generated by the `partition_tdv_grasp()` function.

The simulated annealing algorithm decreases the temperature according to a predefined cooling schedule, which can be controlled by the user by entering the following parameters: the initial temperature (`t_inic`), the final temperature (`t_final`), the fraction of temperature dropping (`alpha`) and the number of iterations `n_iter`. Given these inputs, the cooling schedule is obtained calculating the number of times, say `nt`, that the temperature has to drop (by multiplying the current temperature by `1 - alpha`) in order to approximate `t_final` starting from `t_inic`. The number of times that the temperature has to drop (`nt`) is calculated by the expression: `floor(log(t_final / t_inic) / log(1 - alpha))`. Finally, these decreasing stages are scattered through the desired iterations `n_iter` homogeneously, by calculating the indices of the iterations that will experience a decrease in temperature using `floor(n\_iter / nt * (1:nt))`.

To illustrate the use of `optim_tdv_simul_anne()` and in order to reproduce the example in the original article of Portela-Pereira et al. (2021), we start by removing taxa occurring in only one relevé:

```
taxus_bin_wmt <- taxus_bin[rowSums(taxus_bin) > 1, ]
```

We then apply the simulated annealing algorithm to this new data, starting from a uniformly-generated initial partition and searching for three groups. We selected five runs keeping the progress of the optimization for all of them (by setting `n_sol = 5` and `full_output = TRUE`), so that we can inspect the convergence of the method. We use `set.seed()` only to ensure the repeatability of the outputs:

```
set.seed(1)
result_sa <- optim_tdv_simul_anne(
 m_bin = taxus_bin_wmt,
 k = 3,
 p_initial = "random",
 n_runs = 5,
 n_sol = 5,
 use_grasp = FALSE,
 full_output = TRUE
 )
```

The maximum TDV obtained in each run can be extracted from the output in the following way:

```
sapply(result_sa$SANN, function(x) x$tdv)
```

```
#> [1] 0.2005789 0.1958471 0.1879158 0.1749407 0.1583337
```

The second highest TDV (0.1958471) corresponds to the partition in three groups (Estrela, Gerês and Galicia) from the original article of Portela-Pereira et al. (2021):

```
result_sa$SANN[[2]]$par
```

```
#>  [1] 3 3 3 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
```

The initial temperature defaults to 0.3 and the final temperature to 10^{-6}. Generally, convergence failure can be spotted when final TDV values are similar to the initial ones, especially when starting from random partitions. In such cases, as a rule of thumb, it is advisable to decrease `t_final` by a factor of 10. Figure 2 shows the plots of the TDV kept in each iteration, in each of the five runs, showing that the method is converging to higher values. Such a plot can be obtained in the following way:

```
plot(
 result_sa$SANN[[1]]$current.tdv,
 type = "l",
 xlab = "Iteration number",
 ylab = "TDV"
 )
for (run in 2:5) {
 lines(result_sa$SANN[[run]]$current.tdv)
 }
```

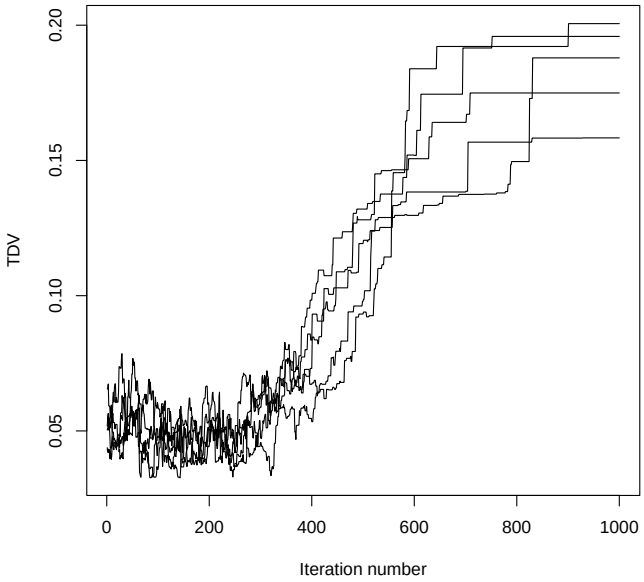

Figure 2: Plots of the TDV obtained in each of the five runs of the simulated annealing optimization.

Table 2: Computational performance of GRASP and simulated annealing (SA) for different numbers of groups k, applied to the *T. baccata* forests data set.

k	GRASP: total time	GRASP: memory allocation	GRASP: TDV	SA: total time	SA: memory allocation	SA: TDV
2	89.42ms	37.66MB	0.1188258	3.08s	1.04GB	0.1284921
3	166.9ms	58.21MB	0.1345673	3.58s	1.31GB	0.1956457
4	328.35ms	78.19MB	0.1991826	4.51s	1.58GB	0.2544701
5	508.1ms	98.01MB	0.2442543	5.1s	1.85GB	0.3183166

We can plot the sorted phytosociological table (see Figure 3), using the same partition as the original work of Portela-Pereira et al. (2021):

```
tabul <- tabulation(
 m_bin = taxus_bin_wmt,
 p = result_sa$SANN[[2]]$par,
 taxa_names = rownames(taxus_bin_wmt),
 plot_im = "normal",
 palette = "Zissou 1"
 )
```

Simulated annealing is often seen as an exploratory technique where the temperature settings are challenging and dependent on the problem. The formulation here implemented (see Section 4) aims at mitigating such a challenge by restricting the temperature values in the interval $]0, 1]$.

This section concludes with a comparison of the computational performance of the GRASP and simulated annealing heuristics. Experiments are conducted on the *T. baccata* forests data set. The number of groups is varied from $k = 2$ to $k = 5$ in both `partition_tdv_grasp()` and `optim_tdv_simul_anne()` using default values for all optional parameters (except `p_initial = "random"` and `use_grasp = FALSE` for the simulated annealing).

Table 2 reports the running time, memory usage, and objective value (TDV) for each method. Running time and memory allocation were obtained with function `mark()` from package **bench** (Hester and Vaughan, 2021).

Simulated annealing consistently achieved higher TDV values for all k, although it required higher memory and running times than those of GRASP. Overall, these results indicate a trade-off between solution quality and computational efficiency, with simulated annealing providing more accurate solutions at the expense of longer computation times.

6 Concluding remarks

In this paper, we presented the mathematical approaches implemented in the R package **diffval** for maximizing TotDiffVal, a recently proposed criterion for assessing the quality of partitions of

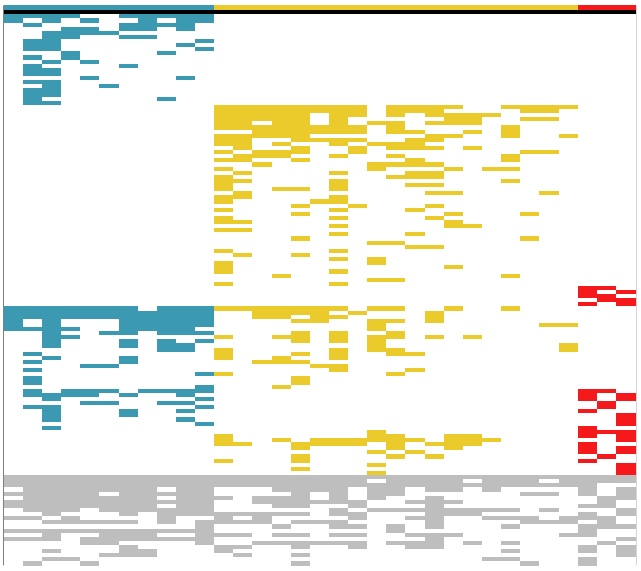

Figure 3: Image of the sorted phytosociological matrix, showing the three groups obtained by the simulated annealing optimization.

the columns (samples or objects) of a binary (0/1) matrix by capturing the exclusiveness of species (features, variables, or attributes) (Monteiro-Henriques, 2025).

We proved that finding a partition into M subsets that maximizes TotDiffVal is NP-hard for any fixed M. For the case $M = 2$, we provided two alternative integer linear programming formulations, both implemented in **diffval**, which allow optimal solutions to be obtained for moderately sized data sets. For larger instances and for $M > 2$, we developed two heuristic algorithms: a Greedy Randomized Adaptive Search Procedure (GRASP) and a Simulated Annealing approach. Good-quality partitions can often be achieved by running the Simulated Annealing algorithm initialized with solutions obtained from multiple runs of the GRASP. However, for very large instances, both heuristics may require excessive computation times. It should be noted that computing the TotDiffVal index (2) for a given M-partition requires a number of operations on the order of the matrix size ($m \times n$), and that re-evaluating the index after small modifications entails a similar computational burden. When the above procedures become computationally very expensive, an alternative is to use the function `partition_tdv_grdtp()`, which is also included in the **diffval** package. This function implements a simplified version of the Greedy algorithm that operates in the following way: Firstly, M columns are selected randomly to work as seeds for each one of the desired M groups. Secondly, one of the remaining columns is selected randomly and added to the partition group which maximizes the upcoming TDV. This second step is repeated until all columns are placed in a group of the M-partition. This function performs faster than `partition_tdv_grasp()` and `optim_tdv_simul_anne()`, but at the cost of the quality of the solutions produced.

Alternative clustering and partitioning methods especially designed for vegetation data are available, see e.g., the **vegclust** (De Cáceres, 2025) package (where k-means and partition around the medoids are modified into fuzzy clustering approaches) and the **vegan** (Oksanen et al., 2025) package (where several specific distance measures are implemented to use in agglomerative clustering). A methodological comparative analysis with 18 clustering/partitioning methods is provided in Monteiro-Henriques (2025). While most of these methods are not as computationally demanding as maximizing TDV, it should be emphasized that TDV optimization fundamentally differs from most clustering and partitioning approaches in three key aspects: i) it does not rely on hierarchical structures or geometric (distance-based) searches, ii) it treats species occurrences as not equally informative, and iii) it values a degree of exclusiveness among informative species. Encoding properties (i–iii) in a computational framework inevitably entails substantial computational costs, a challenge that the **diffval** package addresses by providing a range of exact and heuristic procedures, allowing users to balance solution quality and computation time according to their specific datasets.

7 Appendix

7.1 Computational complexity of maximizing TotDiffVal

We start by polynomially transforming the problem of finding a minimum bisection of a graph to the problem of maximizing the criterion TotDiffVal for $M = 2$.

Let $G = (V, E)$ be a graph with $|V|$ even. A bisection of G is a partition of the vertex set V into two sets of equal sizes. The size of a bisection is the number of edges going across the two sets. A minimum bisection of G is a bisection of V with minimal size. The size of a minimum bisection, denoted by $bw(G)$, is called the bisection width of graph G. The problem of finding a minimum bisection is NP-hard (Garey et al., 1976).

Let $B = [b_{ij}]$ be the transpose of the incidence matrix of graph $G = (V, E)$, i.e., $b_{ij} = 1\,(0)$ if vertex $v_j \in V$ and edge $e_i \in E$ are (not) incident. Add to B two columns of zeros. Let 0 and $n+1$ denote the two new columns and denote by B_0 the resulting $m \times (n+2)$ matrix, where $m = |E|$ and $n = |V|$ is even.

Note that if $S = (G_1, G_2)$ is an arbitrary 2-partition of the set of $n+2$ columns of B_0, with $|G_1| = |G_2| = \frac{n+2}{2}$, and if i is an arbitrary row, then

$$\text{DiffVall}_i(S) = \begin{cases} \frac{4}{n+2} & \text{if both vertices of edge } e_i \text{ belong to the same set of the 2-partition } S \\ 0 & \text{otherwise.} \end{cases}$$

If columns 0 and $n+1$ are in different sets of the 2-partition S, $\text{TotDiffVal}(S) = \sum_{i=1}^{m} \text{DiffVal}_i(S) = \frac{4}{n+2}(m - bw(S))$, where $bw(S)$ is the size of the bisection of G corresponding to S, i.e., $(G_1 \setminus \{0, n+1\}, G_2 \setminus \{0, n+1\})$. If P denotes the set of all 2-partitions of the set of columns of matrix B_0 in two sets of equal size that have columns 0 and $n+1$ in different sets, and if
$bw(G)$ denotes the bisection width of graph G, we have

$$bw(G) = m - \frac{n+2}{4} \max_{S \in P} \text{TotDiffVal}(S). \tag{26}$$

Hence, we may find a minimum bisection of G by maximizing $\text{TotDiffVal}(S)$, for 2-partitions $S \in P$.

We now create a $2n \times (n+2)$ matrix, B', with the same column indices as B_0, and vertically append a number of copies of B' to B_0 to obtain a matrix which is such that the 2-partitions of the set of columns that maximize TotDiffVal are precisely the balanced partitions that have columns 0 and $n+1$ belonging to different sets.

Matrix $B' = [b'_{ij}]$, as referred above, has the same column indices as B; $b'_{i0} := 1, i = 1, \ldots, n$; $b'_{in+1} := 1, i = n+1, \ldots, 2n$; $b'_{ii} = b'_{n+ii} := 1, i = 1, \ldots, n$ and all the remaining entries are equal to zero. In fact, B' is the transpose of the incidence matrix of the complete bipartite graph $K_{2,n}$ with bipartite classes $\{0, n+1\}$ and V.

Let $\bar{B}$ denote the $(m + 2n\delta) \times (n+2)$ matrix obtained by vertically appending to B_0 δ copies of B'.

We will prove that, for $\delta = n^3$,

i) If $S = (G_1, G_2)$ is a 2-partition of the set of columns of $\bar{B}$ that maximizes TotDiffVal, and that has columns 0 and $n+1$ belonging to different sets, then S is balanced (i.e. $|G_1| = |G_2| = \frac{n+2}{2}$).

ii) If $S' = (G'_1, G'_2)$ is a 2-partition of the set of columns of $\bar{B}$ that maximizes TotDiffVal, and that has columns 0 and $n+1$ belonging to the same set, say $0, n+1 \in G'_1$, then $|G'_1| = n+1$ and $|G'_2| = 1$.

iii) $\text{TotDiffVal}(S) > \text{TotDiffVal}(S')$.

From i), ii) and iii) we can conclude that, for $\delta = n^3$, if $S = (G_1, G_2)$ is a 2-partition of the set of columns of $\bar{B}$ that maximizes TotDiffVal, then columns 0 and $n+1$ belong to different sets, and the bisection of G corresponding to S, i.e., $(G_1 \setminus \{0, n+1\}, G_2 \setminus \{0, n+1\})$ is a minimum bisection of graph G. Since the size of matrix $\bar{B}$ is polynomial on the size of graph G ($\text{size}(\bar{B}) = O(n^3)$), we have the following result.

Proposition 1 *Finding an M-partition that maximizes* TotDiffVal *is NP-hard, for* $M = 2$. □

We now prove i), ii) and iii).

Lemma 2 *For* $\delta = n^3$, *if* $S = (G_1, G_2)$ *is a 2-partition of the set of columns of* $\bar{B}$ *that maximizes* TotDiffVal, *and that has columns* 0 *and* $n+1$ *belonging to different sets, then S is balanced (i.e.* $|G_1| = |G_2| = \frac{n+2}{2}$*).*

Proof. Let $X = (X_1, X_2)$ be a 2-partition of the set of columns of $\bar{B}$ with 0 and $n+1$ belonging to

different sets. Suppose $|X_1| = t < \frac{n+2}{2}$.

$$\text{TotDiffVal}(X) = \frac{2m_1}{t} + \frac{2m_2}{n+2-t} + 2\delta\left(\frac{t-1}{t} + \frac{n+1-t}{n+2-t}\right), \tag{27}$$

where m_1 (m_2) is the the number of edges of the subgraph of graph G induced by the vertices of $X_1 \setminus \{0, n+1\}$ ($X_2 \setminus \{0, n+1\}$).

Let X' be a 2-partition obtained by moving an arbitrary column $j \neq 0, n+1$, from X_2 (the largest set) to X_1.

$$\text{TotDiffVal}(X') = \frac{2(m_1 + |E_j(G_1)|)}{t+1} + \frac{2(m_2 - |E_j(G_2)|)}{n+1-t} + 2\delta\left(\frac{t}{t+1} + \frac{n-t}{n+1-t}\right),$$

where $E_j(G1)$ ($E_j(G2)$) is the set of edges linking vertex j with vertices of G_1 (G_2). We can therefore conclude that

$$\text{TotDiffVal}(X') \geq \frac{2m_1}{t+1} + \frac{2(m_2 - (n-t))}{n+1-t} + 2\delta\left(\frac{t}{t+1} + \frac{n-t}{n+1-t}\right). \tag{28}$$

From (27) and (28) we have

$$\text{TotDiffVal}(X') - \text{TotDiffVal}(X) \geq 2\frac{-m_1 p + (m_2 - (n-t)(n+2-t))t(t+1) + \delta(p - t(t+1))}{t(t+1)p},$$

where $p = (n+1-t)(n+2-t)$.

From $t < \frac{n}{2}$, we have $p - t(t+1) > 2(t+1)$ and since $m_1 < \frac{t(t-1)}{2}$, we can conclude that

$$\text{TotDiffVal}(X') - \text{TotDiffVal}(X) > 2\frac{-t(t-1)p/2 + (m_2 - p)t(t+1) + 2\delta(t+1)}{t(t+1)p}.$$

For $\delta = n^3$ the right hand side of the previous inequality is positive and the result follows. □

Lemma 3 *For $\delta = n^3$, if $S' = (G'_1, G'_2)$ is a 2-partition of the set of columns of $\bar{B}$ that maximizes* TotDiffVal*, and that has columns 0 and $n+1$ belonging to the same set, say $0, n+1 \in G'_1$, then $|G'_1| = n+1$ and $|G'_2| = 1$.*

Proof. Let $X = (X_1, X_2)$ be a 2-partition of the set of columns of $\bar{B}$ with 0 and $n+1$ belonging to the same set. Suppose $0, n+1 \in X_1$ and $|X_1| = t < n-1$.

$$\text{TotDiffVal}(X) = \frac{2m_1}{t} + \frac{2m_2}{n+2-t} + 4\delta\,\frac{t-2}{t}, \tag{29}$$

where m_1 (m_2) is the the number of edges of the subgraph of graph G induced by the vertices of $X_1 \setminus \{0, n+1\}$ (X_2).

Let X' be a 2-partition obtained by moving an arbitrary column j from X_2 to X_1.

$$\text{TotDiffVal}(X') = \frac{2(m_1 + |E_j(G'_1)|)}{t+1} + \frac{2(m_2 - |E_j(G'_2)|)}{n+1-t} + 4\delta\,\frac{t-1}{t+1},$$

where $E_j(G'_1)$ ($E_j(G'_2)$) is the set of edges linking vertex j with vertices of G'_1 (G'_2). We can therefore conclude that

$$\text{TotDiffVal}(X') \geq \frac{2m_1}{t+1} + \frac{2(m_2 - (n+1-t))}{n+1-t} + 4\delta\,\frac{t-1}{t+1}. \tag{30}$$

From (29) and (30) we have

$$\text{TotDiffVal}(X') - \text{TotDiffVal}(X) \geq 2\frac{-m_1 p + m_2 t(t+1) - pt(t+1) + 4\delta p}{t(t+1)p},$$

where $p = (n+1-t)(n+2-t)$.

Since $m_1 \leq \frac{(t-2)(t-3)}{2}$ and $t < n-1$, for $\delta = n^3$, the right hand side of the previous inequality is positive and the result follows. □

Lemma 4 *For $\delta = n^3$, if $S = (G_1, G_2)$ and $S' = (G'_1, G'_2)$ are as in Lemma 2 and Lemma 3, respectively, then* TotDiffVal$(S) >$ TotDiffVal(S').

Proof. From (26) we have

$$\text{TotDiffVal}(S) = \frac{4}{n+2}(m - bw(G)) + 4\delta\frac{n}{n+2},$$

where $bw(G)$ is the bisection width of graph G.

We now use the upper bound on $bw(G) \leq \frac{mn}{2(n-1)}$ (Schmidt, 2017, p. 3) to write

$$\text{TotDiffVal}(S) \geq \frac{4m}{n+2}\left(1 - \frac{n}{2(n-1)}\right) + 4\delta\frac{n}{n+2}.$$

For the 2-partition S' the following upper bound holds

$$\text{TotDiffVal}(S') \leq \frac{2m}{n+1} + 4\delta\frac{n-1}{n+1}.$$

From the two previous expressions we have

$$\text{TotDiffVal}(S) - \text{TotDiffVal}(S') \geq \frac{-4mn + 8\delta(n-1)}{(n+2)(n+1)(n-1)}.$$

Since $m \leq \frac{n(n-1)}{2}$, for $\delta = n^3$, the right hand side of the previous inequality is positive and the result follows. □

We now extend Proposition 1 to an arbitrary number of groups.

Proposition 5 *Finding an M-partition that maximizes* TotDiffVal *is NP-hard, for any fixed number of groups* $M \geq 2$.

Proof. We use induction on M. We show that maximizing the criterion TotDiffVal, for arbitrary $M \geq 2$, polynomially transforms to the problem of maximizing TotDiffVal, for $M+1$ groups.

Suppose, for a given 0/1 matrix A, $m \times n$, we want to maximize TotDiffVal, for M groups. Add to A $3m$ zero rows (rows $m+1, \ldots, m+3m$), and append to the resulting matrix an all one column as the rightmost column (column $n+1$). Let $\bar{A}$ be the $4m \times (n+1)$ matrix thus obtained. Let S and S' be arbitrary $(M+1)$-partitions of $\bar{A}$ such that, in partition S, column $n+1$ is the unique column in a group, while in partition S', column $n+1$ belongs to a group of size $t > 1$. Clearly, $\text{TotDiffVal}(S) \geq 3m$ and $\text{TotDiffVal}(S') \leq m + \frac{3}{t}m$, which implies that, for $t > 1$, $\text{TotDiffVal}(S) > \text{TotDiffVal}(S')$. Hence, column $n+1$ is isolated in a group in every optimal $(M+1)$-partition of $\bar{A}$. Consequently, the set of the other M groups of columns, which form an optimal M-partition of the first n columns of $\bar{A}$, is also an optimal M-partition of matrix A.

The fact that, for $M = 2$, the result holds (Proposition 1), completes the proof. □

Acknowledgments

JOC is funded by national funds through FCT – Fundação para a Ciência e a Tecnologia, I.P., under the scope of the projects UID/00297/2025 (https://doi.org/10.54499/UID/00297/2025) and UID/PRR/00297/2025 (https://doi.org/10.54499/UID/PRR/00297/2025) (Center for Mathematics and Applications – NOVA Math). TMH was partially funded by the European Social Fund (POCH and NORTE 2020) and National Funds (MCTES) through Fundação para a Ciência e a Tecnologia, I.P., postdoctoral fellowship (SFRH/BPD/115057/2016), and by National Funds through the FCT - Fundação para a Ciência e a Tecnologia, I.P., under the project UIDB/04033/2020.

References

E. Aarts, J. Korst, and P. Laarhoven, van. Simulated annealing. In E. Aarts and J. Lenstra, editors, *Local search in combinatorial optimization*, pages 91–120. Wiley-Interscience, 1997. ISBN 0-471-94822-5. URL https://pure.tue.nl/ws/portalfiles/portal/1332973/496713.pdf. [p335]

J. S. Borrero, C. Gillen, and O. A. Prokopyev. Fractional 0–1 Programming: Applications and Algorithms. *Journal of Global Optimization*, 69(1):255–282, September 2017. URL https://doi.org/10.1007/s10898-016-0487-4. [p334]

H. Bruelheide. A new measure of fidelity and its application to defining species groups. *Journal of Vegetation Science*, 11:167–178, 2000. URL https://doi.org/10.2307/3236796. [p331]

H. Bruelheide and T. Flintrop. Arranging phytosociological tables by species-relevé groups. *Journal of Vegetation Science*, 5(3):311–316, 1994. ISSN 11009233, 16541103. URL http://www.jstor.org/stable/3235854. [p331]

E. N. Castanho, H. Aidos, and S. C. Madeira. Biclustering data analysis: a comprehensive survey. *Briefings in Bioinformatics*, 25(4):bbae342, 07 2024. ISSN 1477-4054. doi: 10.1093/bib/bbae342. URL https://doi.org/10.1093/bib/bbae342. [p331]

M. De Cáceres. *vegclust: Fuzzy Clustering of Vegetation Data*, 2025. URL https://CRAN.R-project.org/package=vegclust. R package version 2.0.3. [p340]

M. Dufrêne and P. Legendre. Species assemblages and indicator species: The need for a flexible asymmetrical approach. *Ecological Monographs*, 67(3):345–366, 1997. ISSN 00129615. URL https://doi.org/10.2307/2963459. [p331]

T. A. Feo and M. G. Resende. Greedy randomized adaptive search procedures. *Journal of global optimization*, 6(2):109–133, 1995. URL https://doi.org/10.1007/BF01096763. [p335]

E. Feoli and L. Orlóci. Analysis of concentration and detection of underlying factors in structured tables. *Vegetatio*, 40(1):49–54, 1979. ISSN 00423106. URL http://www.jstor.org/stable/20145697. [p331]

P. Festa and M. G. C. Resende. Grasp. In R. Martí, P. M. Pardalos, and M. G. C. Resende, editors, *Handbook of Heuristics*, pages 465–488. Springer International Publishing, Cham, 2018. ISBN 978-3-319-07124-4. URL https://doi.org/10.1007/978-3-319-07124-4_23. [p335]

M. Garey, D. Johnson, and L. Stockmeyer. Some simplified np-complete graph problems. *Theoretical Computer Science*, 1(3):237–267, 1976. ISSN 0304-3975. URL https://doi.org/10.1016/0304-3975(76)90059-1. [p333, 341]

M. R. Garey and D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness (Series of Books in the Mathematical Sciences)*. W. H. Freeman, first edition edition, 1979. ISBN 0716710455. URL https://www.bibsonomy.org/bibtex/2fac25cd952418baa739dd35243b3dbdc/folke. [p333]

Gurobi Optimization, LLC. Gurobi Optimizer Reference Manual, 2023. URL https://www.gurobi.com. [p336]

J. O. Hanson, R. Schuster, N. Morrell, M. Strimas-Mackey, B. P. M. Edwards, M. E. Watts, P. Arcese, J. Bennett, and H. P. Possingham. *prioritizr: Systematic Conservation Prioritization in R*, 2022. URL https://CRAN.R-project.org/package=prioritizr. R package version 8.0.6. [p336]

J. Hester and D. Vaughan. *bench: High Precision Timing of R Expressions*, 2021. URL https://CRAN.R-project.org/package=bench. R package version 1.1.2. [p336, 339]

M. O. Hill. *TWINSPAN - A FORTRAN program for arranging multivariate data in an ordered two-way table by classification of the individuals and attributes*. Section of Ecology and Systematics, Ithaca, New York, USA, 1979. [p331]

L. Kaufman and P. J. Rousseeuw. *Finding groups in data: an introduction to cluster analysis*. Wiley series in probability and mathematical statistics. Wiley, New York, 1990. ISBN 978-0-471-87876-6. URL https://doi.org/10.1002/9780470316801. [p331]

S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220 (4598):671–680, 1983. doi: 10.1126/science.220.4598.671. URL https://www.science.org/doi/abs/10.1126/science.220.4598.671. [p335]

T. Monteiro-Henriques. TDV-optimization: A novel numerical method for phytosociological tabulation. *Vegetation Classification and Survey*, 6:99–127, 2025. URL https://doi.org/10.3897/VCS.140466. [p331, 332, 340]

T. Monteiro-Henriques and J. O. Cerdeira. *diffval: Vegetation Patterns*, 2025. URL https://pointveg.gitlab.io/diffval/. R package version 1.2.0. [p333]

D. Mueller-Dombois and H. Ellenberg. *Aims and Methods of Vegetation Ecology*. Wiley, 1974. ISBN 978-0-471-62290-1. URL https://books.google.pt/books?id=Gu8sAQAAMAAJ. [p331]

J. Oksanen, G. L. Simpson, F. G. Blanchet, R. Kindt, P. Legendre, P. R. Minchin, R. O'Hara, P. Solymos, M. H. H. Stevens, E. Szoecs, H. Wagner, M. Barbour, M. Bedward, B. Bolker, D. Borcard, T. Borman, G. Carvalho, M. Chirico, M. De Caceres, S. Durand, H. B. A. Evangelista, R. FitzJohn, M. Friendly, B. Furneaux, G. Hannigan, M. O. Hill, L. Lahti, C. Martino, D. McGlinn, M.-H. Ouellette, E. Ribeiro Cunha, T. Smith, A. Stier, C. J. Ter Braak, and J. Weedon. *vegan: Community Ecology Package*, 2025. URL https://CRAN.R-project.org/package=vegan. R package version 2.7-2. [p340]

R. K. Peet and D. W. Roberts. *Classification of Natural and Semi-natural Vegetation*, chapter 2, pages 28–70. John Wiley & Sons, Ltd, 2013. ISBN 978-1-118-45259-2. URL https://doi.org/10.1002/9781118452592.ch2. [p331]

J. Podani and E. Feoli. A general strategy for the simultaneous classification of variables and objects in ecological data tables. *Journal of Vegetation Science*, 2(4):435–444, 1991. URL https://doi.org/10.2307/3236025. [p331]

E. a. M. Portela-Pereira, T. Monteiro-Henriques, C. Casas, N. Forner, I. Garcia-Cabral, J. a. P. Fonseca, and C. Neto. Teixedos no noroeste da península ibérica. *Finisterra*, 56(117):127–150, Set. 2021. URL https://doi.org/10.18055/Finis18102. [p336, 338, 339]

J. Roleček, L. Tichý, D. Zelený, and M. Chytrý. Modified TWINSPAN classification in which the hierarchy respects cluster heterogeneity. *Journal of Vegetation Science*, 20(4):596–602, Aug. 2009. ISSN 1654-1103. doi: 10.1111/j.1654-1103.2009.01062.x. URL http://onlinelibrary.wiley.com/doi/10.1111/j.1654-1103.2009.01062.x/abstract. [p331]

T. J. Schmidt. *On the minimum bisection problem in tree-like and planar graphs – structural and algorithmic results*. PhD thesis, Technische Universität München, Germany, 2017. URL https://mediatum.ub.tum.de/doc/1338548/1338548.pdf. [p343]

J. Orestes Cerdeira
Center for Mathematics and Applications (NOVA Math)
NOVA School of Science and Technology (NOVA FCT), Universidade NOVA de Lisboa
2829-516 Caparica, Portugal
ORCiD: 0000-0002-3814-7660
jo.cerdeira@fct.unl.pt

Tiago Monteiro-Henriques
Global Change and Conservation Lab (GCC)
Faculty of Biological and Environmental Sciences, University of Helsinki
Viikinkaari 1, P.O. Box 65, 00014 University of Helsinki, Finland
ORCiD: 0000-0002-4206-0699
tmh.dev@icloud.com